Taller: Observabilidad básica de microservicios (Spring Boot Actuator + Micrometer)

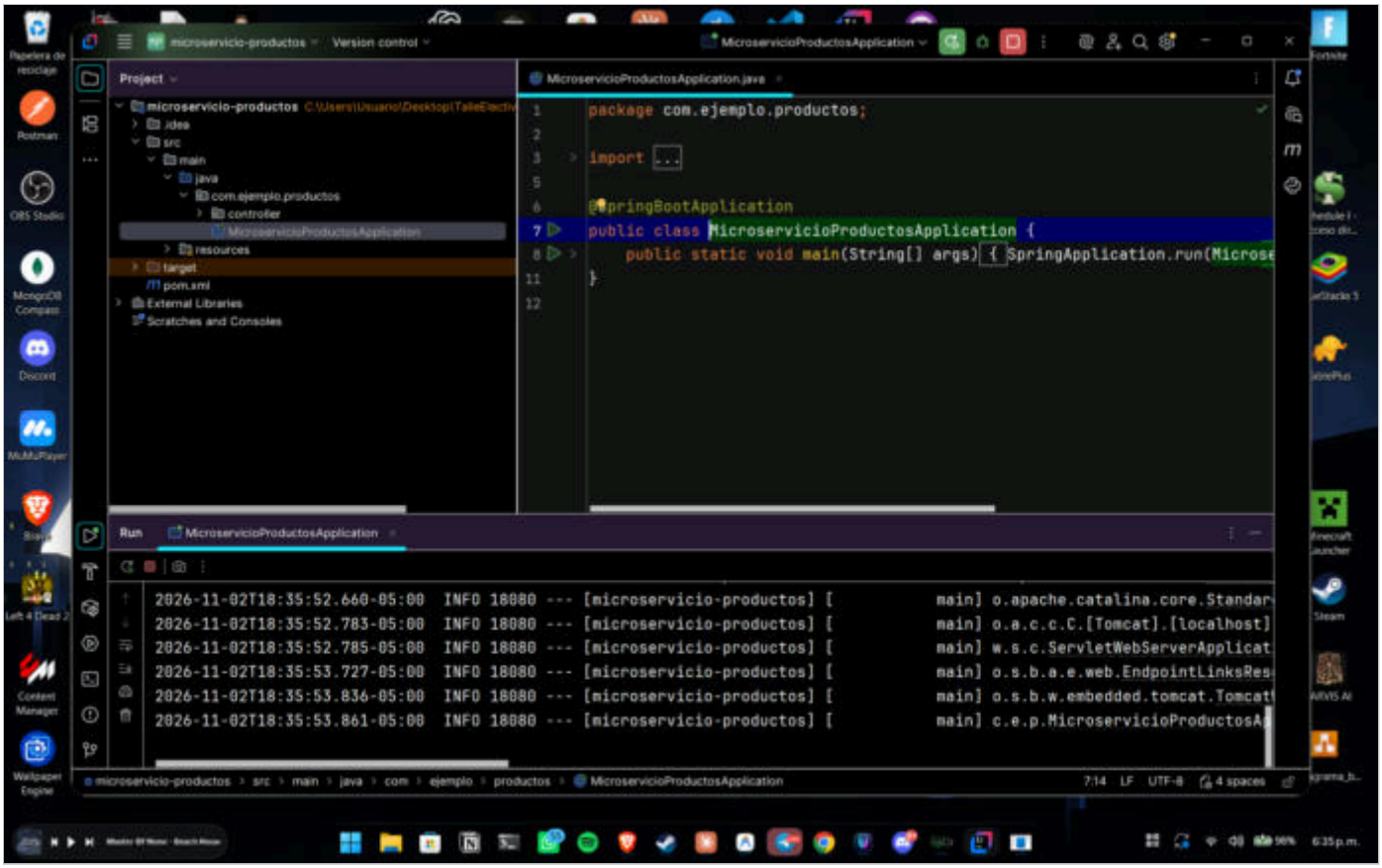
Asignatura: Electiva 1

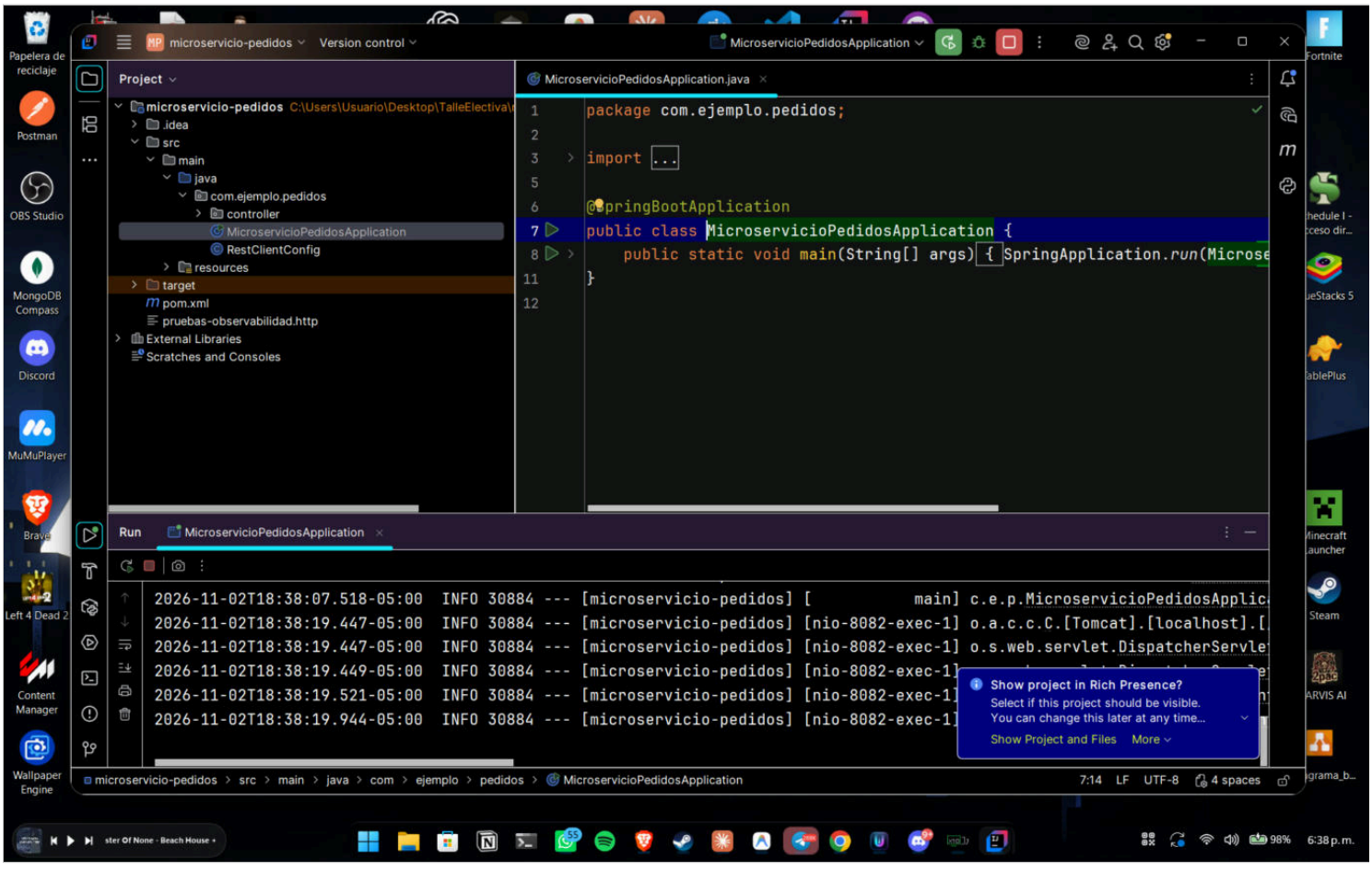
Integrantes:

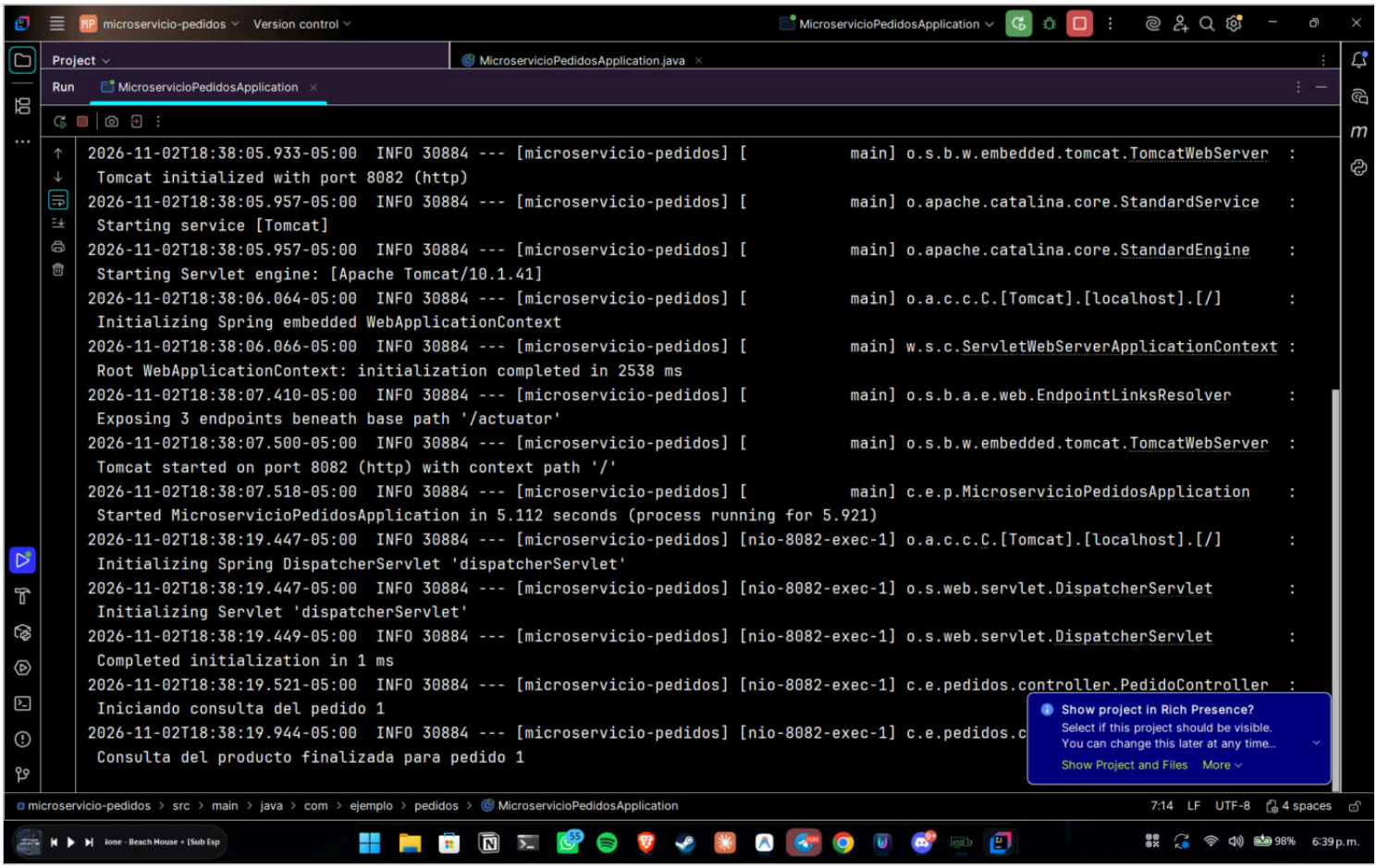
- Carlos Mario Bechara Arias
- Jose Daniel Zambrano Luna
- Rafael Eduardo Sarmiento Peña

1. Ejecución de ambos microservicios

- `microservicio-productos` → puerto 8081 (`/productos/{id}`)
- `microservicio-pedidos` → puerto 8082 (`/pedidos/{id}`), llama a productos con `RestClient`
- Ambos corren simultáneamente (Java 21, Spring Boot 3.5.0, Maven).

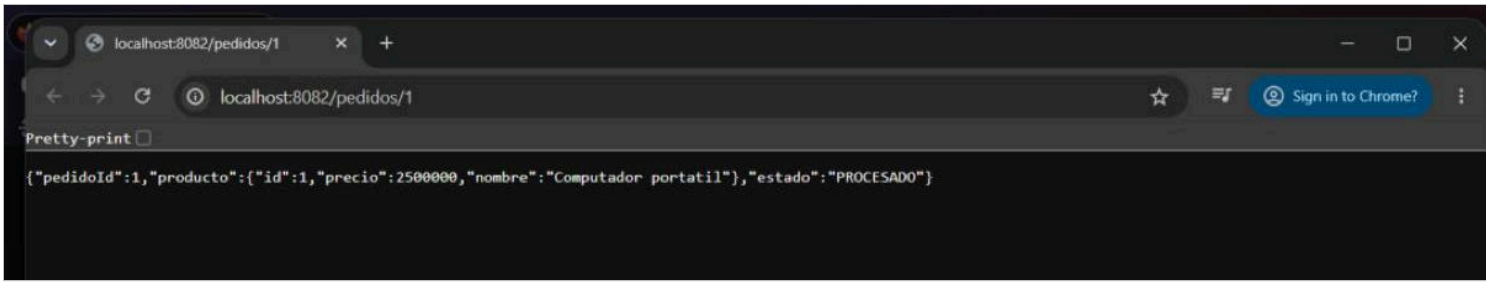

microservicio-productos
Version control
MicroservicioProductosApplication
Project
MicroservicioProductosApplication.java
package com.ejemplo.productos;
import ...
@SpringBootApplication
public class MicroservicioProductosApplication {
public static void main(String[] args) { SpringApplication.run(Microse
}
Run
MicroservicioProductosApplication
2026-11-02T18:35:52.660-05:00 INFO 18080 --- [microservicio-productos] [main] o.apache.catalina.core.Standar
2026-11-02T18:35:52.783-05:00 INFO 18080 --- [microservicio-productos] [main] o.a.c.c.C.[Tomcat].[localhost]
2026-11-02T18:35:52.785-05:00 INFO 18080 --- [microservicio-productos] [main] w.s.c.ServletWebServerApplicat
2026-11-02T18:35:53.727-05:00 INFO 18080 --- [microservicio-productos] [main] o.s.b.a.e.web.EndpointLinksRes
2026-11-02T18:35:53.836-05:00 INFO 18080 --- [microservicio-productos] [main] o.s.b.w.embedded.tomcat.Tomcat
2026-11-02T18:35:53.861-05:00 INFO 18080 --- [microservicio-productos] [main] c.e.p.MicroservicioProductosA
microservicio-productos > src > main > java > com > ejemplo > productos > MicroservicioProductosApplication
7:14 LF UTF-8 4 spaces


microservicio-pedidos
Version control
MicroservicioPedidosApplication
Project
MicroservicioPedidosApplication.java
microservicio-pedidos C:\Users\Usuario\Desktop\TalleElectiva\
.idea
src
main
java
com.ejemplo.pedidos
controller
MicroservicioPedidosApplication
RestClientConfig
resources
target
pom.xml
pruebas-observabilidad.http
External Libraries
Scratches and Consoles
package com.ejemplo.pedidos;
import ...
@SpringBootApplication
public class MicroservicioPedidosApplication {
public static void main(String[] args) { SpringApplication.run(Microse
}
Run
MicroservicioPedidosApplication
2026-11-02T18:38:07.518-05:00 INFO 30884 --- [microservicio-pedidos] [main] c.e.p.MicroservicioPedidosApplic
2026-11-02T18:38:19.447-05:00 INFO 30884 --- [microservicio-pedidos] [nio-8082-exec-1] o.a.c.c.C.[Tomcat].[localhost].[
2026-11-02T18:38:19.447-05:00 INFO 30884 --- [microservicio-pedidos] [nio-8082-exec-1] o.s.web.servlet.DispatcherServle
2026-11-02T18:38:19.449-05:00 INFO 30884 --- [microservicio-pedidos] [nio-8082-exec-1]
2026-11-02T18:38:19.521-05:00 INFO 30884 --- [microservicio-pedidos] [nio-8082-exec-1]
2026-11-02T18:38:19.944-05:00 INFO 30884 --- [microservicio-pedidos] [nio-8082-exec-1]
Show project in Rich Presence?
Select if this project should be visible.
You can change this later at any time...
Show Project and Files
More
microservicio-pedidos > src > main > java > com > ejemplo > pedidos > MicroservicioPedidosApplication
7:14 LF UTF-8 4 spaces



2. Respuesta de `/pedidos/1` (HTTP 200)

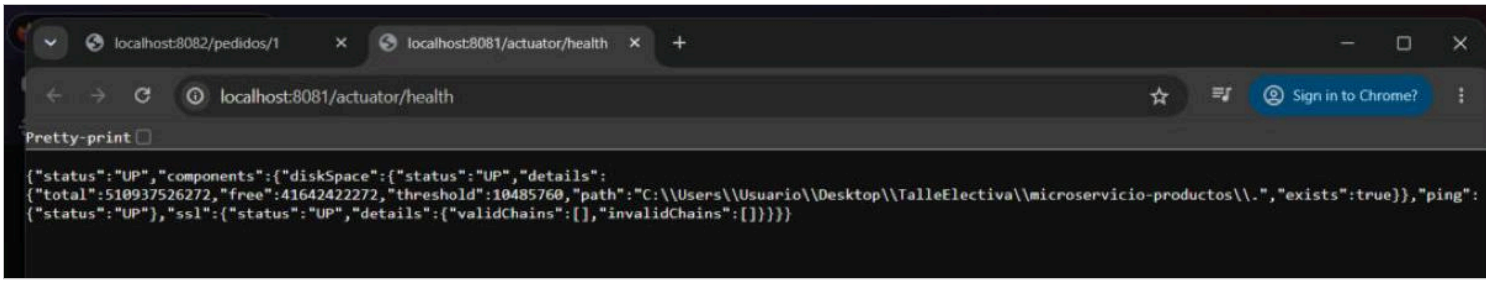

```
{"producto":{"nombre":"Computador portatil","precio":2500000,"id":1},"estado":"PROCESADO","pedidoId":1}
```

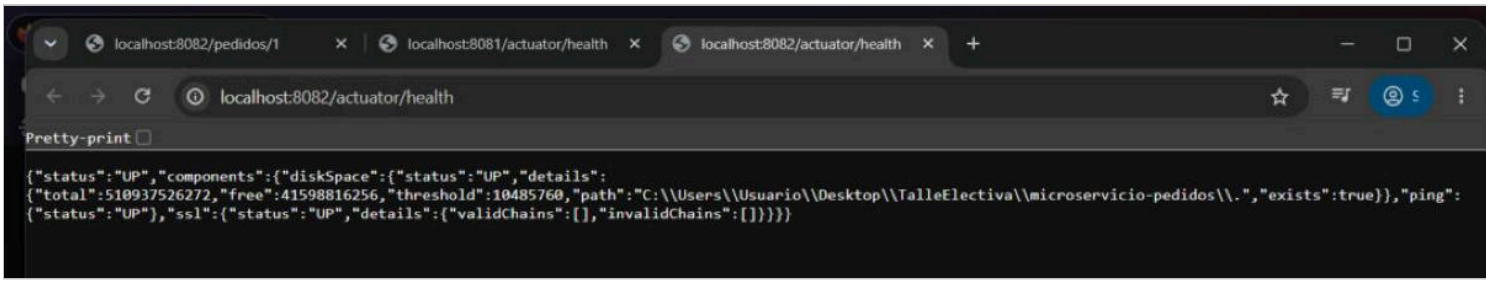



Log productos: `Consultando producto con ID 1`. Log pedidos: `Iniciando consulta del pedido 1` / `Consulta del producto finalizada para pedido 1`.

3. `/actuator/health` de ambos servicios

Ambos devuelven `{"status":"UP", components: diskSpace UP, ping UP, ssl UP}`.





4. Experimento 1: métricas `http.server.requests`

Los tres valores provienen de una misma ejecución desde IntelliJ (pedidos reiniciado para poner los contadores en cero), filtrando por `uri:/pedidos/{id}` en pedidos y `uri:/productos/{id}` en productos.

Servicio	Momento	COUNT	TOTAL_TIME (s)	MAX (s)
Pedidos (8082)	Antes: 1 solicitud	1	0.1974	0.1974
Pedidos (8082)	Después: 10 solicitudes adicionales	11	0.2733	0.1974
Productos (8081)	Después	13*	0.1066	0.0818

* Productos no se reinició, por eso acumula 2 llamadas previas a la medición (una prueba directa de verificación y una consulta anterior al reinicio de pedidos): 11 de pedidos + 2 = 13.

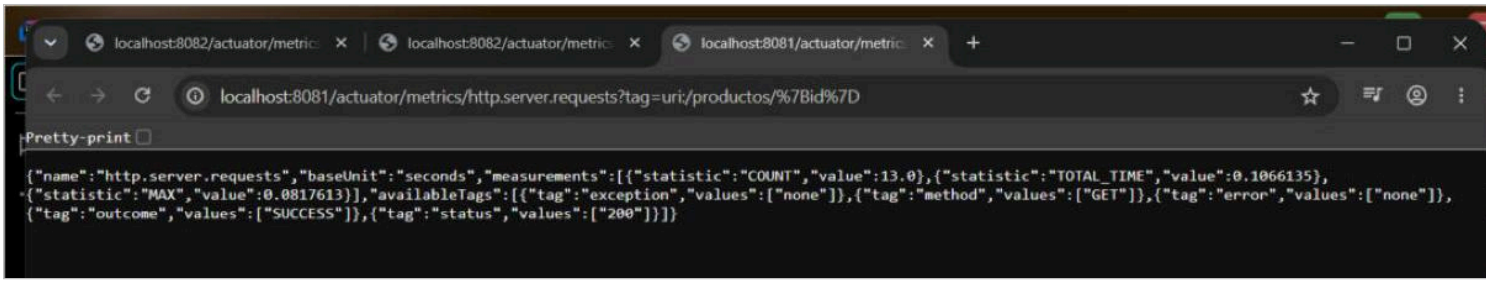
Antes (pedidos):



Después (pedidos):



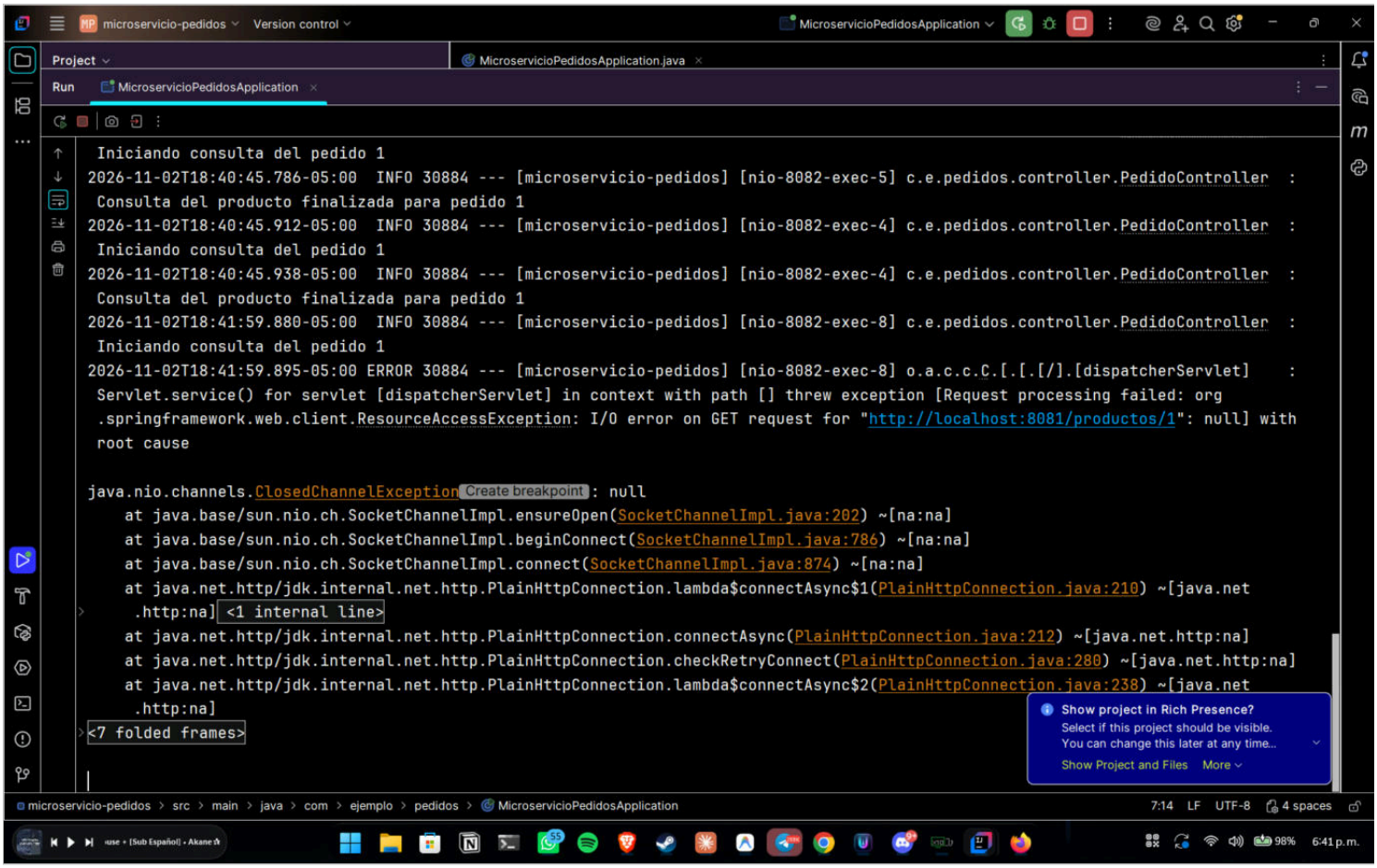
Después (productos):



Interpretación: - COUNT pasó de 1 a 11: las 10 solicitudes adicionales quedaron registradas. - TOTAL_TIME pasó de 0.1974 s a 0.2733 s: las 10 solicitudes nuevas sumaron solo ≈0.076 s (≈7.6 ms cada una), mientras la primera tardó ≈197 ms (arranque en frío: carga de clases y primera conexión). - MAX se mantuvo en 0.1974 s porque ninguna solicitud posterior superó a la primera. - Tiempo promedio = TOTAL_TIME / COUNT: bajó de 197 ms a ≈25 ms por el efecto del arranque en frío. - Pedidos tarda más que productos porque su operación incluye la llamada HTTP a productos.

5. Experimento 2: caída de productos

- Con productos detenido, `GET /pedidos/1` → **HTTP 500 Internal Server Error**.
- En la consola de pedidos aparece el `ERROR ... Servlet.service() ... threw exception [Request processing failed: ... ResourceAccessException: I/O error on GET request for "http://localhost:8081/productos/1"]`.
- `GET /actuator/health` de pedidos siguió en **UP**.





- Al reiniciar productos, `/pedidos/1` volvió a responder 200 PROCESADO.

¿Permanece UP? ¿Por qué? Sí. El health predeterminado solo revisa recursos locales (disco, ping, ssl) y no comprueba servicios HTTP remotos, así que pedidos está "sano" aunque su operación de negocio falle.

6. Preguntas de análisis

1. **COUNT vs TOTAL_TIME:** COUNT es el número de peticiones registradas; TOTAL_TIME es la suma de sus duraciones. TOTAL_TIME/COUNT da el tiempo promedio.
2. **¿Por qué pedidos está UP con productos caído?** Porque su health solo evalúa el estado local del proceso; la dependencia remota no forma parte del chequeo por defecto.
3. **Datos para detectar degradación:** tiempo promedio (TOTAL_TIME/COUNT) y MAX crecientes, tasa de respuestas 5xx/errores (tags `status`, `outcome`, `exception`), métricas JVM (memoria, GC, hilos) y CPU, y logs de errores/timeouts.
4. **Aporte de una traza distribuida:** sigue una misma petición a través de varios servicios con un trace id, mostrando en qué servicio y tramo se pierde tiempo o falla, algo que el log aislado de cada servicio no correlaciona.
5. **Riesgos de exponer `/actuator/metrics` y `/health` con detalles:** revelan información interna (rutas, versiones, rutas de disco, estado de componentes, uso de recursos) útil para un atacante; deben restringirse con autenticación/red interna y ocultar detalles.

7. Conclusión

- **Métricas:** valores numéricos agregados en el tiempo (conteos, tiempos); indican *que* algo está lento o fallando. **Logs:** eventos textuales puntuales de cada servicio; explican *qué* ocurrió. **Trazas:** recorrido de una petición entre servicios; muestran *dónde* ocurrió.
- **Limitación del health predeterminado:** solo refleja la salud local; un servicio puede figurar UP mientras sus operaciones fallan por una dependencia caída. Se requiere health indicators de dependencias, timeouts y monitoreo de métricas de errores.